

Why Java 8 Streams Processing?

More expressive, declarative and powerful way to work with collections



Why care about Java 8?

- ✓ Java-style functional programming
- ✓ JSR 335 – Lambda Expression Closure
- ✓ JEP 107 – Bulk data operation for collections
forEach, filter, map, reduce
- ✓ Support for parallel processing
- ✓ More concise and readable code
- ✓ Brings functional flavors to Java



What is the offer?

“.....Is a blend of imperative and object oriented programming enhanced with functional flavors”

Java DNA



Evolution of Java

- Java 7** Fork and join
- Java 8** Parallel processing
Declarative data processing
- Classical threads** vs JUC
- Concurrency** vs parallel processing
- SQL** vs Java
- Optimization** should be Job of JVM or programmer?



Methods vs. Functions

Aspect	Method	Pure Function
Mutation	A method mutates , i.e., it modifies data and produces other changes and side effects.	A pure function does not mutate it just inspects data and produces results in form of new data.
How vs. What	Methods describe how things are done.	Pure functions describe what is done rather than how it is done.
Invocation Order	Methods are used imperatively order of invocation matters.	Pure functions are used declaratively.



Imperative vs. Declarative

Imperative (Traditional)

```
List<String> result = new ArrayList<>();
for (String s : list) {
    if (s.startsWith("A")) {
        result.add(s.toUpperCase());
    }
}
```

Declarative (Streams)

```
List<String> result = list.stream()
    .filter(s -> s.startsWith("A"))
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

Streams provide a declarative way to express bulk data operations for collections (forEach, filter, map, reduce)



Key Takeaways



Functional programming support
(Lambdas, Streams)



Efficient bulk data operations
(forEach, filter, map, reduce)



Parallel processing made easier



More expressive and readable code



Brings functional flavors to Java while keeping OOP and imperative strengths

“.....Is a blend of imperative and object oriented programming enhanced with functional flavors”

Functional Interface in Java 8

A new term in Java 8 to enable functional programming (Lambdas, Streams, etc.)



What is a Functional Interface?

- New term of Java 8.
- A functional interface is an interface with only one *abstract* method.

```
public interface Runnable {  
    run();  
}
```

```
public interface Comparator<T> {  
    int compare(T t1, T t2);  
}
```



Interface Default and Static Methods

- Java 8 extends interface declarations with two new concepts:
 - Default methods
 - Static methods
- Advantages:
 - No longer need to provide your own companion utility classes. Instead, you can place static methods in the appropriate interfaces.
 - Adding methods to an interface without breaking the existing implementation.



java.util.function Package

- Brand new java.util.function package.
- Rich set of function interfaces.
- 4 categories:

Supplier

Represents a supplier of results (no input).

Consumer

Represents an operation that accepts a single input.

Predicate

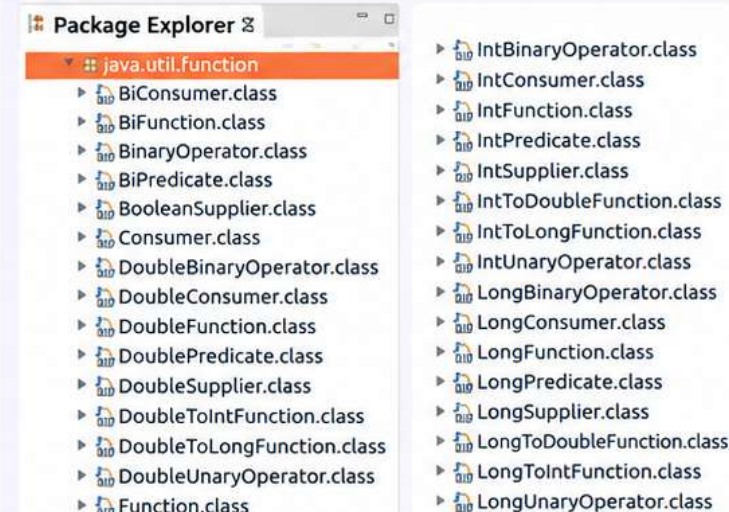
Represents a predicate (boolean-valued function).

Function

Represents a function that accepts an input and produces a result.



Available Interfaces (Examples)



Example – Static Method in Interface

```
java.util.Collections }  
java.util.Collection }  
static <T> Collection<T>  
synchronizedCollection(  
    Collection<T> c)
```

No longer need to create a separate utility class.
Static methods can be placed in the interface itself.



Why Functional Interface?

- ✓ Enables Java-style functional programming (Lambdas, Streams).
- ✓ Makes code more concise and readable.
- ✓ Provides a rich set of ready-to-use interfaces in java.util.function.
- ✓ Supports modern features like forEach, filter, map, reduce.
- ✓ Helps add new methods to interfaces (default/static) without breaking existing code.

Conclusion: Functional interfaces are a key building block in Java 8 for functional programming and the Stream API.

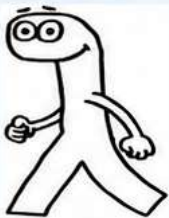
Lambda Expression in Java 8

A concise way to represent a single abstract method implementation



? What is Lambda Expression?

- Provide a clear and concise way to represent a “one abstract method interface” (a **functional interface**) using an expression.
- Lambda expression = anonymous method call.



First Class Functions

⚙️ Syntax and Parts

(argument list) → { body of the method }

A lambda is composed of three parts:

Part	Example (from listener)
Argument list	event
Arrow token	->
Body of the method	System.out.println("Click!")

💡 Why it works?

- It works because the listener has only one abstract method and the compiler can infer what to do from the interface.
- Lambda expressions are used where a functional interface (an interface with only one abstract method) is expected.

```
package java.awt.event;
import java.util.EventListener;
public interface ActionListener extends EventListener {
    /**
     * Invoked when an action occurs.
     */
    public void actionPerformed(ActionEvent e);
}
```

</> Example – Button Listener

```
JButton testButton = new JButton("Test Button");
testButton.addActionListener(event -> System.out.println("Click!"));
```

Old style (Anonymous Inner Class)

```
Arrays.sort(testStrings, new Comparator<String<>() {
    @Override
    public int compare(String s1, String s2) {
        return(s1.length() - s2.length());
    }
});
```



New style (Lambda Expression)

```
Arrays.sort(testStrings,
    (s1, s2) -> { return(s1.length() - s2.length()); }
);
```

🎯 When to use Lambda Expression?

- ✓ When you have a functional interface (i.e., only one abstract method).
- ✓ To make code more concise and readable.
- ✓ Commonly used with collections, event handling, threads, streams, etc.

📈 Performance Difference?

- ✓ Lambda expressions are more concise and readable. They can also offer performance benefits in many cases compared to anonymous inner classes.

Key Takeaway: Lambda expressions reduce boilerplate code, improve readability, and make it easier to work with functional interfaces in Java 8.



Write Once
Run Anywhere

Behavior Parameterization in Java 8

Light weight way to implement strategy design pattern

Same Code
Different Behavior ! 😊

1 What is Behavior Parameterization?

- A way to pass behavior (code) as a parameter instead of hardcoding it inside a method.
- In Java 8, it is achieved using lambda expressions and functional interfaces.
- It is a **light weight way to implement strategy design pattern**.

“Same code, different behavior based on the parameter passed !”

2 Our Inventory – List of Apples



3 Lambda Expression?

- New term of Java 8.
- A functional interface is an interface with only one abstract method.

```
public interface Runnable {
    run();
}
```

```
public interface Comparator<T> {
    int compare(T t1, T t2);
}
```

Key Points

- ✓ Lambda expression = anonymous method call.
- ✓ Provides a clear and concise way to represent a “one abstract method interface” (functional interface) using an expression.
- ✓ Compiler can infer what to do because the interface has only one abstract method.

4 Old Style vs New Style (Example)

Old style (Anonymous Inner Class)

```
Arrays.sort(testStrings, new Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return(s1.length() - s2.length());
    }
});
```

New style (Lambda Expression)

```
Arrays.sort(testStrings,
    (s1, s2) -> { return(s1.length() - s2.length()); }
);
```

5 Behavior Parameterization Example

New Behavior

apple.getWeight() > 150

apple.getColor.equals("green")

```
static List<Apple> filterApples(
    List<Apple> inventory, Predicate<Apple> p) {
    List<Apple> result = new ArrayList<>();
    for (Apple apple: inventory) {
        if (p.test(apple)) {
            result.add(apple);
        }
    }
    return result;
}
```

Output

Heavy Apples

Green Apples

6 java.util.function Package

- Brand new java.util.function package.
- Rich set of function interfaces.
- 4 categories:

Supplier	Represents a supplier of results (no input).
Consumer	Represents an operation that accepts a single input.
Predicate	Represents a predicate (boolean-valued function).
Function	Represents a function that accepts an input and produces a result.

7 When to use Lambda Expression?

- ✓ When you have a functional interface (i.e., only one abstract method).
- ✓ To make code more concise and readable.
- ✓ Commonly used with collections, event handling, threads, streams, etc.
- ✓ Helps in implementing strategy design pattern in a simple way.

Same Code, Different Behavior – Power of Java 8 Lambda !

Small Change
Big Flexibility ! 😊





Stream Processing in Java 8

An Introduction

Same Problem
Less Code
More Clarity !

Data Processing Requirement

Return the names of dishes that are low in calories (<400),
Sorted by number of calories



1 Old Approach (Step by Step)

1 Filter foods by calories (< 400)

```
List<Dish> foods = new ArrayList<Dish>();  
for (Dish d: foodsAll) {  
    if (d.getCalories() < 400) {  
        foods.add(d);  
    }  
}
```

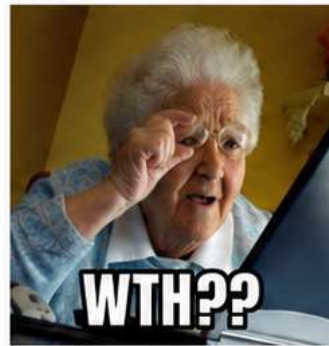
2 Sort by number of calories

```
Collections.sort(foods, new Comparator<Dish>() {  
    @Override  
    public int compare(Dish o1, Dish o2) {  
        return Integer.compare(o1.getCalories(), o2.getCalories());  
    }  
});
```

3 Extract only dish names

```
List<String> names = new ArrayList<String>();  
for (Dish d: foods) {  
    names.add(d.getName());  
}
```

Same code
to achieve the result,
too much boilerplate ?

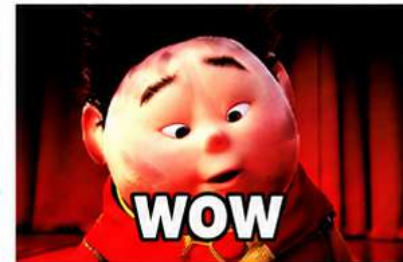


So much code
for a simple task !

2 Java 8 Stream Approach

```
List<String> selectedFoodsNames = foodsAll  
    .stream()  
    .filter(f -> f.getCalories() < 400)  
    .sorted(Comparator.comparing(Dish::getCalories))  
    .map(f -> f.getName())  
    .collect(Collectors.toList());
```

Same result with
clear and concise code !



Lambda expression
= anonymous method call

Behavior Parameterization with Streams
Simpler Code | Better Readability

Write Less
Do More !





Stream Operations in Java 8

Intermediate vs Terminal

Process Data
Step by Step
with Streams !



Intermediate Operations

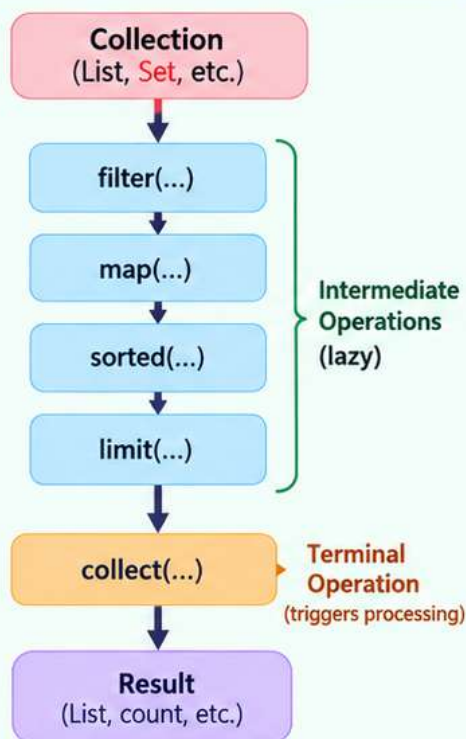
- Return a new Stream (Stream<T> or Stream<R>).
- Are lazy (do not process data immediately).
- Can be chained together.
- Used to transform or filter the data.

Operation	Return Type	Functional Interface	Description
filter	Stream<T>	Predicate<T>	Keeps elements matching a condition (T -> boolean)
map	Stream<R>	Function<T,R>	Transforms each element (T -> R)
flatMap	Stream<R>	Function<T, Stream<R>>	Maps and flattens to a single stream
distinct	Stream<T>	—	Removes duplicate elements
sorted	Stream<T>	Comparator<T>	Sorts elements ((T, T) -> int)
limit	Stream<T>	—	Returns only first n elements
skip	Stream<T>	—	Skips first n elements



Stream Pipeline

Intermediate operations build the pipeline.
Terminal operation triggers the processing.



Terminal Operations

- Produce a result or a side effect.
- Trigger the processing of the pipeline.
- After a terminal operation, the stream is consumed and cannot be reused.

Operation	Return Type	Functional Interface	Description
forEach	void	Consumer<T>	Consumes each element (T -> void)
collect	R	Collector<T, A, R>	Reduces the stream to a collection (List, Set, Map, etc.)
reduce	Optional<T>	BinaryOperator<T>	Combines elements (T, T) -> T
count	long	—	Returns the number of elements
findFirst	Optional<T>	—	Returns the first element (if present)
findAny	Optional<T>	—	Returns any element (if present)
anyMatch	boolean	Predicate<T>	Returns true if any element matches (T -> boolean)
noneMatch	boolean	Predicate<T>	Returns true if no element matches (T -> boolean)
allMatch	boolean	Predicate<T>	Returns true if all elements match (T -> boolean)



Key Takeaway:

- Use intermediate operations to transform and filter the data.
- Use a terminal operation to get the final result.
- Streams provide a clean and concise way to process data compared to external iteration on collections.

Less Code
More Possibilities !



Optional in Java 8

A better way to deal with missing values

Say Goodbye
to `NullPointerException`!

1 The “Billion-Dollar mistake”

- Tony Hoare – the creator of the null reference
- Created while designing ALGOL W
- Meant to model the absence of a value
- “... simply because it was so easy to implement”
- The cause of the dreaded Null Pointer Exceptions

“I call it my billion-dollar mistake.”

– Tony Hoare



2 So what’s wrong with null?



It’s a source of error

► `NullPointerException`



It bloats code

► *Worsens readability*



It’s meaningless

► *Has no semantic meaning*

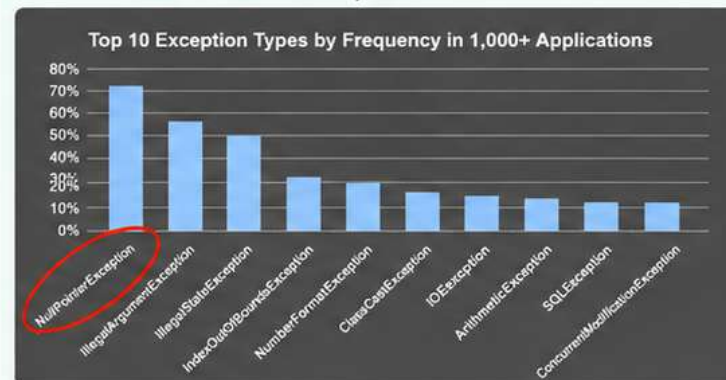


It breaks Java’s philosophy

► *Java hides pointers from developers, except in one case*

3 `NullPointerException`

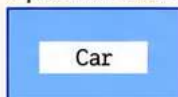
- Thrown when we attempt to use a null value
- The most thrown exception



4 Introducing `Optional`

- New class located at `java.util.Optional<T>`
- Encapsulates optional values
- It wraps the value if it’s there, and is empty if it isn’t
 - `Optional.empty()`
- Signals that a missing value is acceptable
- We can’t dereference `null`, but we can dereference `Optional.empty()`

`Optional<Car>`



Value is present

`Optional<Car>`



Value is empty

5 Key Points

- ✓ `Optional` provides a clear and explicit way to deal with missing values.
- ✓ Helps avoid `NullPointerException`.
- ✓ Improves code readability.
- ✓ Represents the intent that a value may be absent.
- ✓ Use `Optional` instead of returning `null`.

*“We can’t dereference `null`,
but we can dereference `Optional.empty()`”*

6 When to use `Optional`?

- ✓ As a method return type when a value may be absent.
- ✓ To make the absence of a value explicit.
- ✓ To write cleaner and safer code.
- ✓ Not for fields, parameters or where null is perfectly valid.



Remember

`Optional` is not about removing `null` from Java, but about handling missing values in a **better way**.

Less Null | More Clarity | Better Code

Write Clean Java!